

Machine Learning-Based Price Fairness Analysis for Laptop E-Commerce

Manjusha Motamarri

CS5100: Foundations of Artificial Intelligence

GitHub:

<https://github.com/ManjushaMotamarri/Machine-Learning-Based-Price-Fairness-Analysis-for-Laptop-E-Commerce>

1. Introduction

Consumers shopping for laptops online face a persistent and often invisible challenge: it is difficult to determine whether a listed price is reasonable given a laptop's hardware specifications. The laptop market is large, fragmented, and inconsistent in its pricing, with identical or near-identical configurations selling at vastly different prices across brands and retailers. Without deep technical expertise, most consumers poorly evaluate whether they are paying a fair price or being significantly overcharged. This information asymmetry between sellers and buyers is a meaningful consumer protection problem, and one that machine learning is well suited to address.

This project develops a machine learning system that predicts a laptop's expected market price from its hardware specifications and uses that prediction to compute a Price Fairness Score. The score measures the gap between a laptop's actual listed price and its model-predicted price, and uses this gap to classify the listing as Overpriced, Fairly Priced, or Underpriced. The goal is to make pricing judgment transparent, data-driven, and accessible to everyday consumers. While prior work has focused on price prediction accuracy alone, this project extends that foundation by treating prediction error as a fairness signal rather than simply a model flaw.

Prior work in machine learning-based price prediction demonstrates the feasibility of this approach. Balti et al. [1] developed an end-to-end pipeline for predicting laptop prices in the Tunisian market using Gradient Boosting, achieving 94% predictive accuracy. Their work highlighted the importance of rigorous data cleaning and correlation-based feature selection, and demonstrated that tree-based ensemble methods are particularly well-suited to structured hardware tabular data. Kuchkorov et al. [2] conducted a comparative study evaluating Linear Regression, Random Forest, and XGBoost on a Kaggle laptop dataset, finding that XGBoost achieved an R square of 0.93. Their feature importance analysis identified

RAM capacity, storage type, and screen resolution as the strongest predictors of price, findings that directly informed the feature engineering decisions in this project.

However, both studies treat price prediction as an end in itself, optimizing solely for accuracy without considering what prediction error reveals about real-world pricing behavior. Neither work applies their models to any consumer-facing evaluation or asks whether a given listing is priced fairly relative to its hardware value. This project addresses that gap by reframing prediction residuals as a fairness signal - the difference between what a laptop costs and what its specifications predict it should cost. This extension turns a standard regression task into a practical consumer tool, and represents the core contribution of this work.

2. Methods

2.1 Dataset

The dataset used in this project is the Laptop Sales Price Prediction Dataset sourced from Kaggle [\[6\]](#). It contains 1019 laptop listings with features covering CPU specifications, GPU type, RAM amount and type, storage capacity and type, display size and resolution, brand, operating system, and price in Indian Rupees (INR). The dataset was selected for its diversity of configurations and price points, and because it is well-structured with consistent column definitions. No additional data scraping was performed, as initial attempts to collect live listings from Amazon, BestBuy, and Newegg were unsuccessful due to aggressive bot protection on those platforms.

2.2 Data Cleaning and Preprocessing

The raw dataset required several cleaning steps before it could be used for modeling. Missing values were identified and imputed using appropriate summary statistics and removed where imputation was not appropriate. Inconsistent text formatting, typographical errors, and whitespace issues were corrected across categorical columns to ensure consistent encoding. The dataset contained no duplicate entries. The price distribution was found to be strongly right-skewed, which is common in consumer electronics markets where a small number of premium products sit far above the median price. A log transformation was applied to the price column to create the target variable `Log_Price`, which produced an approximately normal distribution better suited for regression modeling.

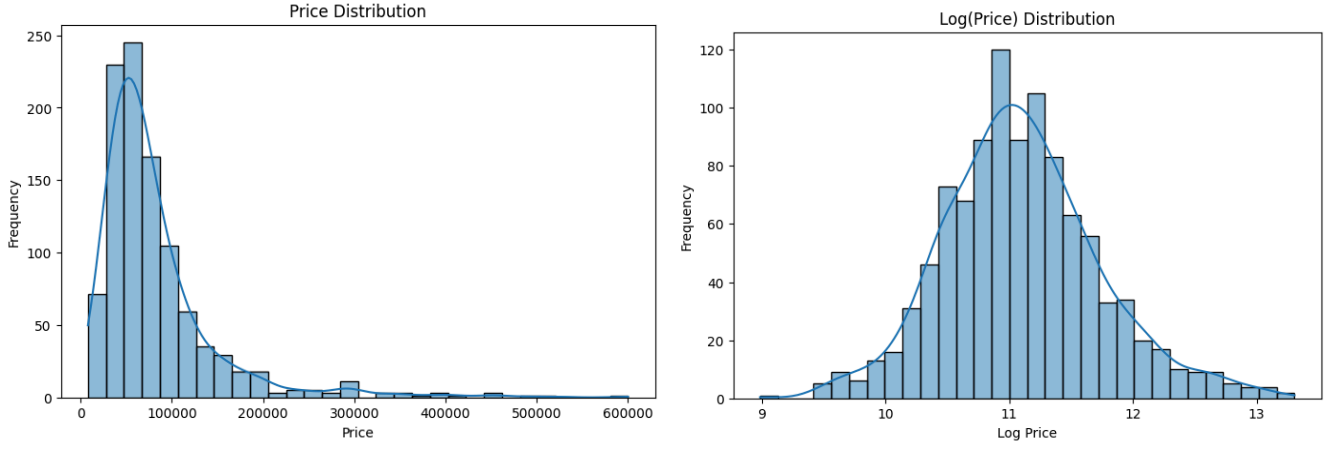


Figure 1: Price distribution before and after log transformation

2.3 Feature Engineering

Feature engineering was a substantial part of this project, as the raw dataset contained several columns that required transformation before they could be used meaningfully in a model. Six low-signal or redundant columns were removed based on near-zero variance or lack of correlation with price. Continuous features including RAM capacity, storage capacity, display size, and processor generation were binned into categorical tiers (Low, Mid, High) to reduce noise, improve robustness to outliers, and capture the non-linear price jumps that occur at key hardware thresholds. For example, the price difference between 8GB and 16GB RAM is far more significant than the difference between 32GB and 64GB, and binning captures this structure more cleanly than treating RAM as a raw numeric value.

Three new features were engineered to capture higher-level information not directly present in any single column. A binary gaming laptop flag (`is_gaming`) was created based on keyword detection in product descriptions. A brand tier variable (`brand_tier`) was derived by computing the average price of each brand and grouping brands into Budget, Mid, Mid-High, and Premium tiers. A composite performance score (`performance_score`) was computed as the product of thread count and processor generation number, intended to capture the combined effect of processor recency and parallelism on overall CPU performance.

Categorical variables were standardized and encoded in two formats: label encoding for tree-based models, where ordinal relationships exist, and one-hot encoding for Linear Regression, where the model cannot infer ordinal structure. This produced two clean datasets of 16 and 58 features respectively.

During initial modeling, tree models were underperforming relative to Linear Regression, which prompted a multicollinearity investigation. Pairwise correlations were computed across all features, and five high-correlation feature pairs were identified with correlation coefficients above 0.8. These included Threads vs. `performance_score` (0.84), GPU_tier vs. Graphics_brand (0.88), and Processor_gen_tier vs.

Processor_brand (0.87). Since performance_score already encodes thread count as part of a composite measure, Threads was dropped in favor of the more informative engineered feature. Similarly, GPU_tier and Processor_gen_tier were retained as more granular representations, and their correlated counterparts Graphics_brand and Processor_brand were removed. The proc_brand_tier column was also dropped due to its high correlation with Processor_gen_tier. After these removals, the final feature set was reduced to 16 features for tree models and 58 for Linear Regression.

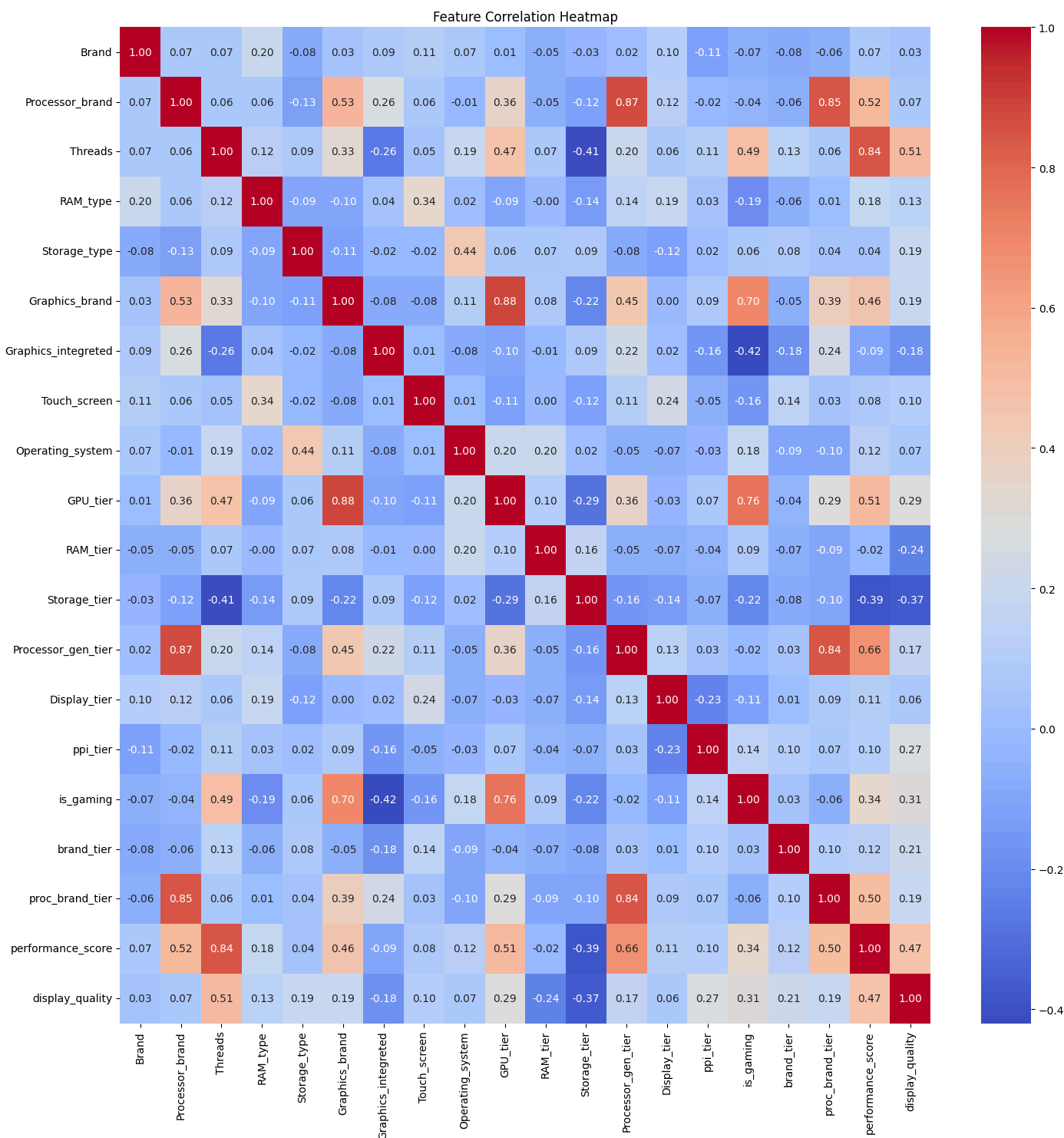


Figure 2: Correlation heatmap of final feature

2.4 Modeling

Three regression models were trained and evaluated using scikit-learn [5]. All models used an 80/20 train-test split with a fixed random seed of 42 to ensure that all models were evaluated on identical test rows, making metric comparisons valid. All metrics were computed on the held-out test set only.

Linear Regression was trained on the one-hot encoded dataset using a scikit-learn Pipeline that applied StandardScaler normalization before fitting. This model served as an interpretable baseline and established a performance floor for the tree-based models to beat.

Random Forest and XGBoost were both trained on the label-encoded dataset and tuned using 5-fold GridSearchCV. The hyperparameter search spaces and best configurations identified are summarized in Table 1. All models were evaluated using root mean squared error (RMSE), mean absolute error (MAE), and the coefficient of determination (R square).

Model	Hyperparameter	Search Space	Best Value
Random Forest	n_estimators	100, 200, 300	300
	max_depth	10, 20, None	20
	min_samples_split	2, 5	2
	max_features	sqrt, log2	sqrt
XGBoost	n_estimators	100, 200, 300	300
	max_depth	3, 5, 7	5
	learning_rate	0.01, 0.05, 0.1	0.05
	subsample	0.8, 1.0	0.8

Table 1. Hyperparameter search space and best values for each model

2.5 SHAP Analysis

To understand which features most influenced the XGBoost model's predictions, SHAP (SHapley Additive exPlanations) analysis was applied using TreeExplainer [3]. SHAP values were computed on the test set and summarized as mean absolute SHAP values to produce a global feature importance ranking.

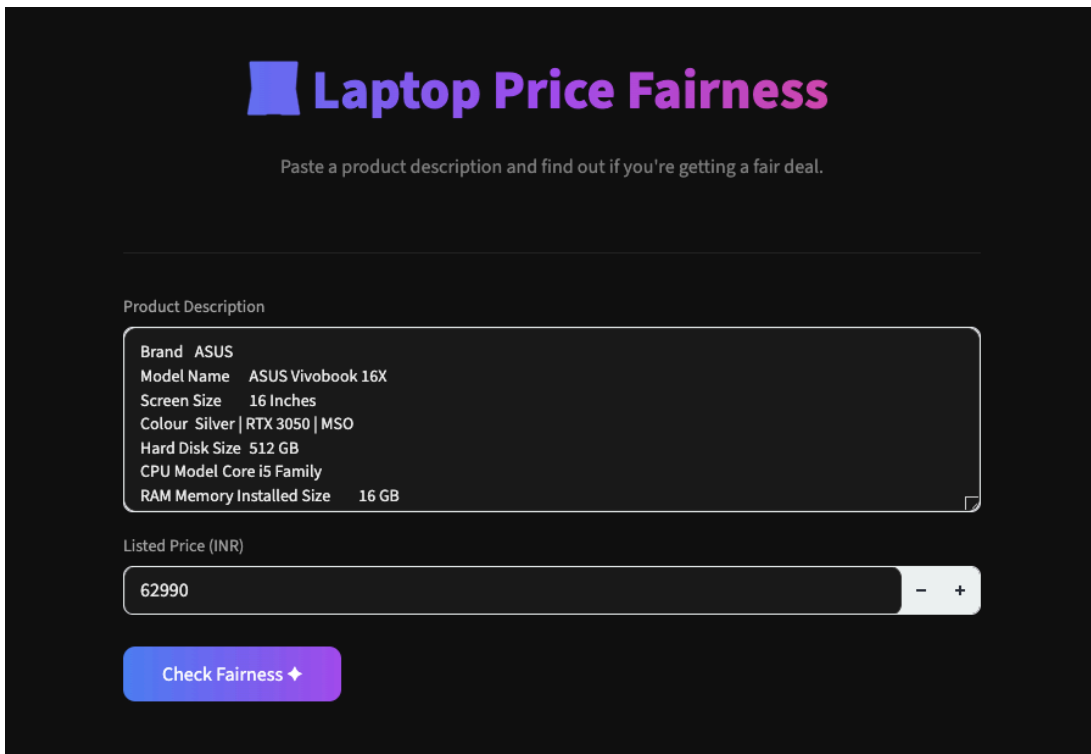
This provided both a validation of the feature engineering decisions and an interpretable explanation of model behavior that could be communicated to non-technical users.

2.6 Price Fairness Score

The Price Fairness Score was computed using predictions from the final XGBoost model applied to the full dataset. Residuals were calculated as the difference between actual and predicted log prices. These were converted to percentage price differences using the inverse log transformation, expressed as $\text{price_diff_pct} = (\exp(\text{residual}) - 1) \times 100$. This quantity directly represents how much more or less a laptop costs compared to what its hardware specifications would predict. A symmetric threshold of +/- 15.45%, corresponding to one standard deviation of the percentage differences across the dataset, was used to classify each laptop as Overpriced, Fairly Priced, or Underpriced.

2.7 Demo Application

A Streamlit web application was developed to make the system interactive and accessible. Users paste a laptop product description in plain text into the app, which extracts relevant hardware features using regex-based pattern matching. These features are encoded using the same label maps used during training and passed to the saved XGBoost model, which returns a predicted price and a fairness verdict.



Laptop Price Fairness

Paste a product description and find out if you're getting a fair deal.

Product Description

Brand ASUS
Model Name ASUS Vivobook 16X
Screen Size 16 Inches
Colour Silver | RTX 3050 | MSO
Hard Disk Size 512 GB
CPU Model Core i5 Family
RAM Memory Installed Size 16 GB

Listed Price (INR)

62990

Check Fairness ↗

Figure 3.1: Streamlit Demo - entering details

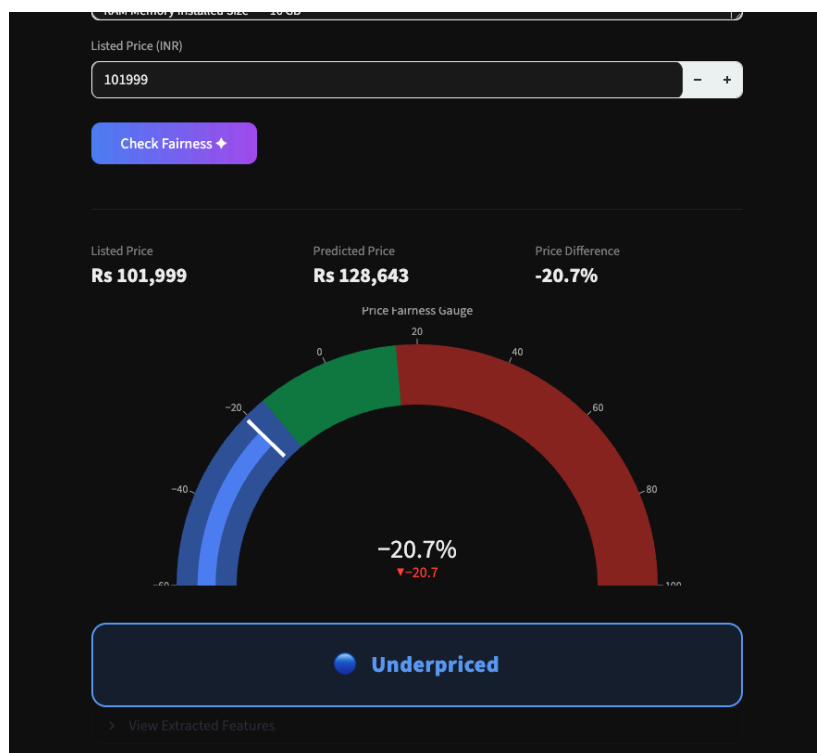


Figure 3.2: Streamlit Demo - Checking fairness of price

3. Results

3.1 Model Performance

The three models performed as follows on the held-out test set. XGBoost performed best across all three metrics and was selected as the final model.

Model	RMSE	MAE	R square
XGBoost (Tuned)	0.2125	0.1418	0.8706
Linear Regression	0.2164	0.1526	0.8657
Random Forest (Tuned)	0.2202	0.1492	0.8610

Table 2. Model performance on a held-out test set (RMSE and MAE in log-price scale).

An R square of 0.87 indicates that the model explains 87% of the variance in laptop prices from hardware specifications alone, which is a strong result for a structured tabular dataset with no seller or market context. The RMSE of 0.2125 in log scale corresponds to approximately a 23% average price error in real terms, meaning that on a Rs 50,000 laptop, the model's prediction is off by roughly Rs 11,500 on average.

A notable finding is that all three models performed within a narrow range of each other, with R square values between 0.86 and 0.87. Linear Regression in particular performed surprisingly close to the tuned tree models, suggesting that the engineered features and log transformation of the target produced relationships between features and price that are largely linear. This is meaningful because it implies that the primary non-linearity in the original data was addressed through binning and feature construction, rather than requiring the model itself to learn complex interactions.

3.2 SHAP Feature Importance

The SHAP analysis revealed that the two engineered features, `performance_score` and `display_quality`, ranked first and second in mean absolute SHAP value, followed by `RAM_tier`, `GPU_tier`, `brand_tier`, and `Storage_tier`. The dominance of engineered features over raw input columns validates the feature engineering decisions made earlier in the pipeline. Features such as `Operating_system`, `Storage_type`, and `Display_tier` had very low SHAP values, indicating minimal contribution to predictions. These could be candidates for removal in a future iteration to further simplify the model.

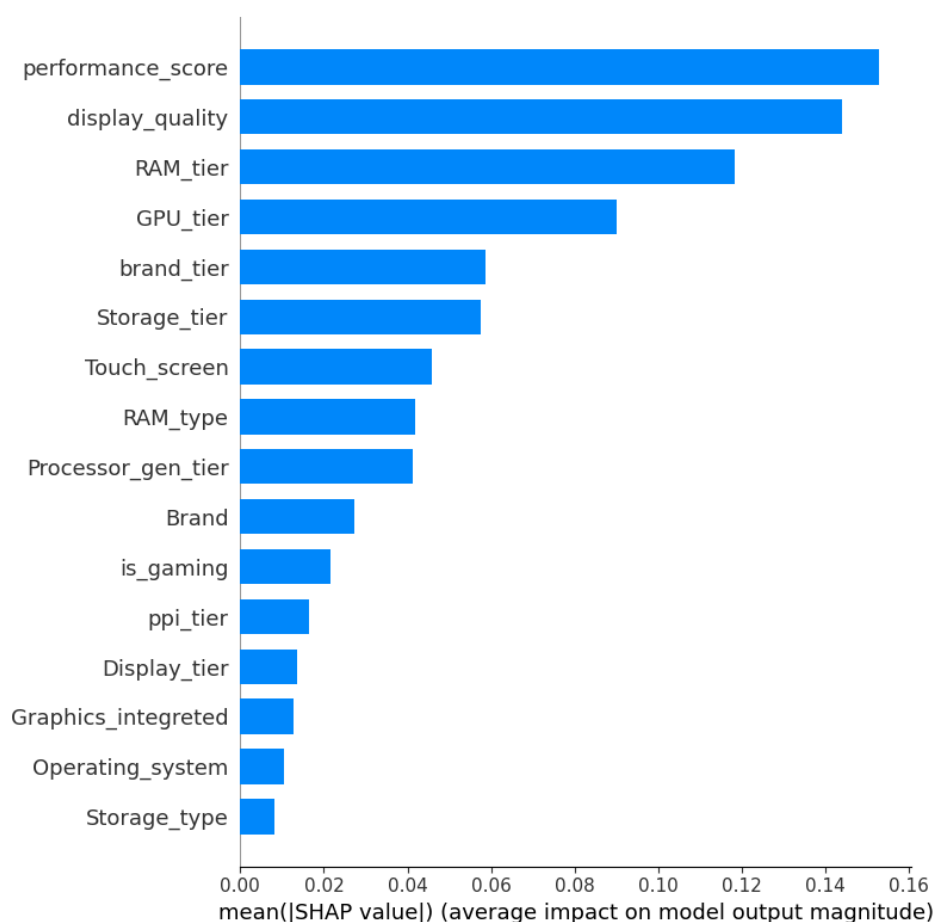


Figure 4: SHAP feature importance bar chart

3.3 Price Fairness Score

Applying the fairness scoring mechanism to the full dataset produced the following distribution of fairness classifications:

Fairness Flag	Count	Percentage
Fairly Priced	879	86.2%
Overpriced	87	8.5%
Underpriced	54	5.3%

Table 3. Distribution of Price Fairness Score classifications across the full dataset.

The distribution of percentage price differences is right-skewed, with a mean of 1.28% and a maximum of 213%. The extreme upper tail suggests a small number of listings where the actual price far exceeds what the hardware specifications would predict, likely reflecting premium brand pricing or niche configurations where brand value is priced well above hardware value. The system successfully identifies these outliers and flags them for consumer attention.

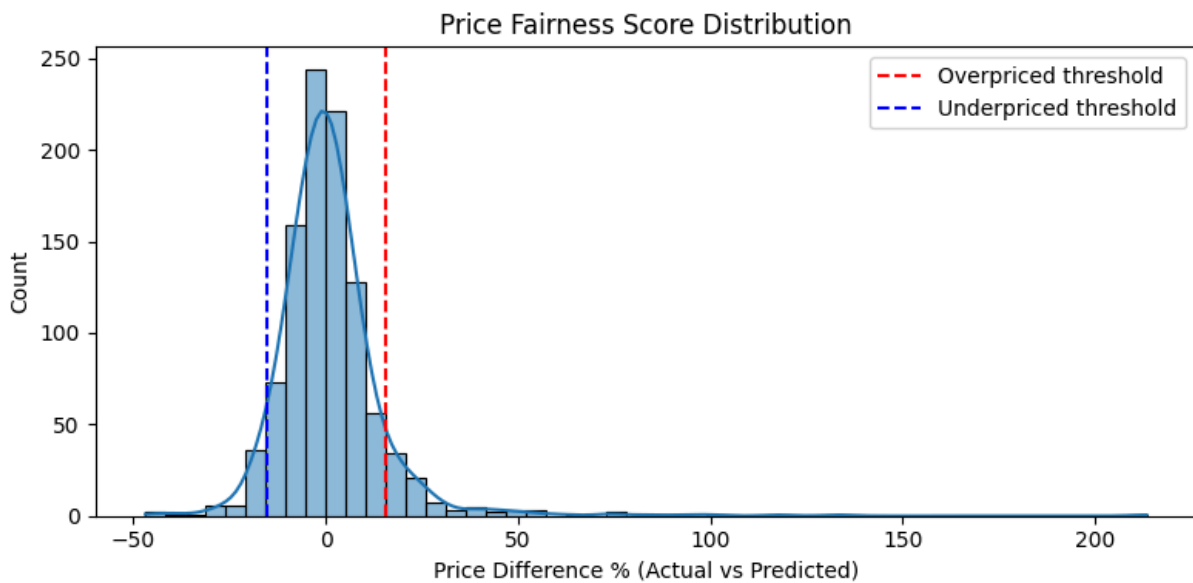


Figure 5: Distribution of percentage price differences with thresholds marked

4. Conclusions

This project successfully developed an end-to-end machine learning pipeline for laptop price prediction and fairness analysis. The tuned XGBoost model achieved an R square of 0.87 on the held-out test set, outperforming Linear Regression (0.87) and Random Forest (0.86). The Price Fairness Score identified 14% of listings as either overpriced or underpriced, and the Streamlit demo makes the system accessible to non-technical users through a simple paste-and-check interface.

The primary strength of this approach is interpretability. SHAP values ground each fairness judgment in specific hardware attributes, making the system transparent and trustworthy for consumer use. The main limitation is that all three models converge around $R^2 = 0.86-0.87$, suggesting a ceiling imposed by the dataset rather than model choice. Factors like seller markup, regional demand, and promotional pricing are absent from hardware specification data alone, and a richer dataset would likely push performance higher. The one-standard-deviation fairness threshold is also a practical but somewhat arbitrary choice that could be refined for specific market segments.

Compared to Balti et al. [1] and Kuchkorov et al. [2], who reported an R^2 of 0.94 and 0.93 respectively, this project achieves slightly lower accuracy, likely due to the deliberate removal of correlated features. The key contribution beyond prior work is the fairness layer itself. Future improvements could include expanding to multiple markets, incorporating live pricing data through an API, and replacing regex-based feature extraction with a large language model for more robust parsing of unstructured product descriptions.

5. Works Cited

- [1] Balti, A., Yassin, M., Lakhoua, M. N., and Sayadi, M. (2023). Predicting Laptop Prices in the Tunisian Market Using Data Mining and Machine Learning Methods. *2023 IEEE Third International Conference on Signal, Control and Communication (SCC), Hammamet, Tunisia*, pp. 1–6. [doi: 10.1109/SCC59637.2023.10527571](https://doi.org/10.1109/SCC59637.2023.10527571)
- [2] Kuchkorov, T., Yusupov, I., Makhkamov, B., Zaynidinov, K., Ather, D., and Pathak, S. (2024). Machine Learning-based Price Prediction and Analysis of Laptop Specifications. *2024 International Conference on Information Science and Communications Technologies (ICISCT), Seoul, Korea*, pp. 352–356. [doi: 10.1109/ICISCT64202.2024.10957051](https://doi.org/10.1109/ICISCT64202.2024.10957051)
- [3] Lundberg, S. M. and Lee, S. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
- [4] Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
- [5] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- [6] Siddiqui, F. (2024). Laptop Sales Price Prediction Dataset. *Kaggle*. kaggle.com/datasets/siddiquifaiznaeem/laptop-sales-price-prediction-dataset-2024/data